

Metody probabilistyczne w uczeniu maszynowym

Wykład 9: boosting, metoda najbliższych sąsiadów,
sieci neuronowe

Katarzyna Grygiel

Semestr letni 2025/2026

Boosting

Słabe klasyfikatory

Nieformalnie: klasyfikator nazwiemy *słabym*, gdy jego działanie jest niewiele lepsze od algorytmu klasyfikującego w sposób zupełnie losowy.

Słabe klasyfikatory

Nieformalnie: klasyfikator nazwiemy *słabym*, gdy jego działanie jest niewiele lepsze od algorytmu klasyfikującego w sposób zupełnie losowy.

Formalnie: algorytm uczący nazwiemy *słabym*, gdy istnieje $\gamma > 0$ taka, że dla dowolnego rozkładu D na danych wejściowych $\mathcal{X}$, dla dowolnej $\delta > 0$ i dla próbki $S \subseteq \mathcal{X}$ o odpowiednio dużym rozmiarze m algorytm zwraca hipotezę h_S , dla której zachodzi nierówność

$$p_{S \sim D^m} \left(R(h_S) \leq \frac{1}{2} - \gamma \right) \geq 1 - \delta.$$

Słabe klasyfikatory

Nieformalnie: klasyfikator nazwiemy *słabym*, gdy jego działanie jest niewiele lepsze od algorytmu klasyfikującego w sposób zupełnie losowy.

Formalnie: algorytm uczący nazwiemy *słabym*, gdy istnieje $\gamma > 0$ taka, że dla dowolnego rozkładu D na danych wejściowych $\mathcal{X}$, dla dowolnej $\delta > 0$ i dla próbki $S \subseteq \mathcal{X}$ o odpowiednio dużym rozmiarze m algorytm zwraca hipotezę h_S , dla której zachodzi nierówność

$$p_{S \sim D^m} \left(R(h_S) \leq \frac{1}{2} - \gamma \right) \geq 1 - \delta.$$

Słabymi klasyfikatorami są np. płytkie drzewa decyzyjne, w szczególności takie o głębokości 1, czyli *pnie* (stumps).

Uczenie grupowe

Ensemble learning (metoda grupowania) polega na łączeniu wielu słabych klasyfikatorów w celu uzyskania jednego dobrego modelu przewidywań.

Uczenie grupowe

Ensemble learning (metoda grupowania) polega na łączeniu wielu słabych klasyfikatorów w celu uzyskania jednego dobrego modelu przewidywań.

Klasyfikatory grupowe możemy podzielić na dwie kategorie:

- równoległe, wykorzystujące niezależność pomiędzy „uczniami” i zmniejszające błąd przez uśrednianie (np. lasy losowe),
- sekwencyjne, bazujące na modyfikowaniu wag zaobserwowanych danych (np. boosting).

Boosting: niedemokratyczna mądrość tłumu

Główna idea boostingu polega na połączeniu T słabych klasyfikatorów h_t w jeden silny klasyfikator h w następujący sposób:

$$h(x) = \text{sgn} \left(\sum_{t=1}^T \alpha_t h_t(x) \right),$$

czyli każdy słaby klasyfikator h_t jest liczony z pewną wagą α_t .

Boosting: niedemokratyczna mądrość tłumu

Główna idea boostingu polega na połączeniu T słabych klasyfikatorów h_t w jeden silny klasyfikator h w następujący sposób:

$$h(x) = \text{sgn} \left(\sum_{t=1}^T \alpha_t h_t(x) \right),$$

czyli każdy słaby klasyfikator h_t jest liczony z pewną wagą α_t .

Ponadto w procesie uczenia każda obserwacja $(x^{(i)}, y^{(i)})$ ze zbioru treningowego również ma ważony wkład $w_t^{(i)}$, zależny od tego, jak dobrze była do tej pory klasyfikowana. Im „trudniejszy” przypadek, tym większą ma wagę.

Boosting: niedemokratyczna mądrość tłumu

Główna idea boostingu polega na połączeniu T słabych klasyfikatorów h_t w jeden silny klasyfikator h w następujący sposób:

$$h(x) = \text{sgn} \left(\sum_{t=1}^T \alpha_t h_t(x) \right),$$

czyli każdy słaby klasyfikator h_t jest liczony z pewną wagą α_t .

Ponadto w procesie uczenia każda obserwacja $(x^{(i)}, y^{(i)})$ ze zbioru treningowego również ma ważony wkład $w_t^{(i)}$, zależny od tego, jak dobrze była do tej pory klasyfikowana. Im „trudniejszy” przypadek, tym większą ma wagę.

Jak wyznaczyć:

- słabe klasyfikatory h_t ?
- ich wagi α_t ?
- wagi obserwacji $w_t^{(i)}$?

Przykładowy boosting: AdaBoost

- Na początku każda z obserwacji ma taką samą wagę:

$$w_1^{(i)} = \frac{1}{m}.$$

Przykładowy boosting: AdaBoost

- Na początku każda z obserwacji ma taką samą wagę:

$$w_1^{(i)} = \frac{1}{m}.$$

- W każdej iteracji t algorytmu ze zbioru klasyfikatorów bazowych wybieramy ten, który na zbiorze obserwacji popełnia najmniejszy błąd ϵ_t . Oznaczamy ten klasyfikator przez h_t i przypisujemy mu wagę

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}.$$

Przykładowy boosting: AdaBoost

- Na początku każda z obserwacji ma taką samą wagę:

$$w_1^{(i)} = \frac{1}{m}.$$

- W każdej iteracji t algorytmu ze zbioru klasyfikatorów bazowych wybieramy ten, który na zbiorze obserwacji popełnia najmniejszy błąd ϵ_t . Oznaczamy ten klasyfikator przez h_t i przypisujemy mu wagę

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}.$$

- Modyfikujemy wagi dla wszystkich obserwacji następująco:

$$w_{t+1}^{(i)} = \begin{cases} w_t^{(i)} \exp(\alpha_t), & \text{gdy } h_t(x^{(i)}) \neq y^{(i)}, \\ w_t^{(i)} \exp(-\alpha_t), & \text{gdy } h_t(x^{(i)}) = y^{(i)}. \end{cases}$$

Przykładowy boosting: AdaBoost cd.

- Wagi obserwacji normalizujemy tak, aby sumowały się do 1.

Przykładowy boosting: AdaBoost cd.

- Wagi obserwacji normalizujemy tak, aby sumowały się do 1.
- Po osiągnięciu warunku stopu ostatecznie zwracamy następujący klasyfikator:

$$h(x) = \text{sgn} \left(\sum_t \alpha_t h_t(x) \right).$$

Przykładowy boosting: AdaBoost cd.

- Wagi obserwacji normalizujemy tak, aby sumowały się do 1.
- Po osiągnięciu warunku stopu ostatecznie zwracamy następujący klasyfikator:

$$h(x) = \text{sgn} \left(\sum_t \alpha_t h_t(x) \right).$$

Możliwe warunki stopu:

- osiągnięcie ustalonej liczby iteracji T ,
- osiągnięcie na zbiorze treningowym przez końcowy klasyfikator h zerowego błędu lub błędu poniżej pewnej ustalonej granicy,
- osiągnięcie przez każdy z klasyfikatorów bazowych błędu ważonego $1/2$.

Algorytm AdaBoost

$\{(x^{(i)}, y^{(i)}): i = 1, \dots, m\}$ – dane

$\mathcal{H}$ – zbiór słabych klasyfikatorów

for $i \leftarrow 1$ **to** m **do**

$w_1^{(i)} \leftarrow \frac{1}{m}$

end

for $t \leftarrow 1$ **to** T **do**

$h_t \leftarrow$ klasyfikator z $\mathcal{H}$ o najmniejszym ważonym błędzie ϵ_t

$\alpha_t \leftarrow \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$

$Z_t \leftarrow 2\sqrt{\epsilon_t(1-\epsilon_t)}$

for $i \leftarrow 1$ **to** m **do**

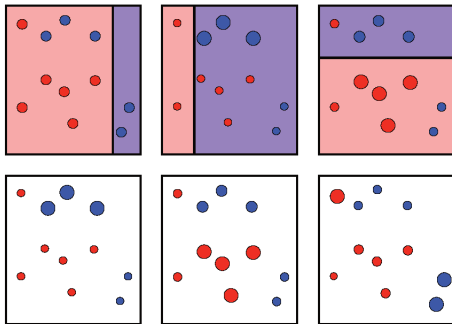
$w_{t+1}^{(i)} \leftarrow \frac{w_t^{(i)}}{Z_t} \exp(-\alpha_t y^{(i)} h_t(x^{(i)}))$

end

end

$h \leftarrow \sum_{t=1}^T \alpha_t h_t$

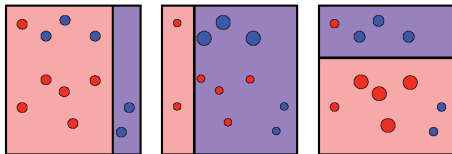
AdaBoost na przykładzie



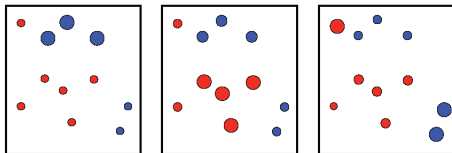
granice decyzyjne

zmiany wag

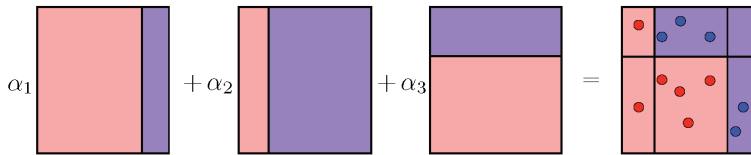
AdaBoost na przykładzie



granice decyzyjne



zmiany wag



ostateczny klasyfikator

AdaBoost jest niezły

Zwiększenie liczby iteracji boostingu sprawia, że błąd empiryczny AdaBoost zmniejsza się wykładniczo.

Twierdzenie

Błąd empiryczny klasyfikatora h zwróconego przez algorytm AdaBoost spełnia

$$\hat{R}_S(h) \leq \exp \left(-2 \sum_{t=1}^T (1/2 - \epsilon_t)^2 \right).$$

Ponadto jeżeli dla każdego $t \in \{1, \dots, T\}$ zachodzi $\gamma \leq 1/2 - \epsilon_t$, to

$$\hat{R}_S(h) \leq \exp(-2\gamma^2 T).$$

Wady i zalety AdaBoost

Zalety:

- jest prosty w implementacji,
- tylko jeden hiperparametr (T),
- może działać dla różnych słabych klasyfikatorów,
- nie ma tendencji do przeuczenia (pokazane eksperymentalnie).

Wady:

- jest podatny na obserwacje odstające,
- może nie działać dobrze w przypadku zbyt słabych lub zbyt złożonych klasyfikatorów bazowych.

Metoda najbliższych sąsiadów

Algorytm k najbliższych sąsiadów

Metoda k najbliższych sąsiadów jest algorytmem nieparametrycznym: nie zakłada niczego o podstawowych danych.

Algorytm k najbliższych sąsiadów

Metoda k najbliższych sąsiadów jest algorytmem nieparametrycznym: nie zakłada niczego o podstawowych danych.

Jest to algorytm leniwy, co oznacza, że nie wymaga żadnej specjalistycznej fazy treningu. Do klasyfikowania nowych punktów korzysta ze wszystkich danych treningowych, zatem uczenie sprowadza się przede wszystkim do przechowywania danych.

Algorytm k najbliższych sąsiadów

Metoda k najbliższych sąsiadów jest algorytmem nieparametrycznym: nie zakłada niczego o podstawowych danych.

Jest to algorytm leniwy, co oznacza, że nie wymaga żadnej specjalistycznej fazy treningu. Do klasyfikowania nowych punktów korzysta ze wszystkich danych treningowych, zatem uczenie sprowadza się przede wszystkim do przechowywania danych.

Można go używać zarówno w problemach klasyfikacji, jak i regresji.

Kiedy wejdiesz między wrony...

Ustalmy $k > 0$.

Aby sklasyfikować nowy punkt $x \in \mathbb{R}^d$ w zbiorze treningowym znajdujemy jego k „najbliższych sąsiadów”, czyli k punktów z $\mathbb{R}^d$ o najmniejszych odległościach.

Następnie punktowi x przypisujemy klasę, do której należy najwięcej „sąsiadów”.

Kiedy wejdziesz między wrony...

Ustalmy $k > 0$.

Aby sklasyfikować nowy punkt $x \in \mathbb{R}^d$ w zbiorze treningowym znajdujemy jego k „najbliższych sąsiadów”, czyli k punktów z $\mathbb{R}^d$ o najmniejszych odległościach.

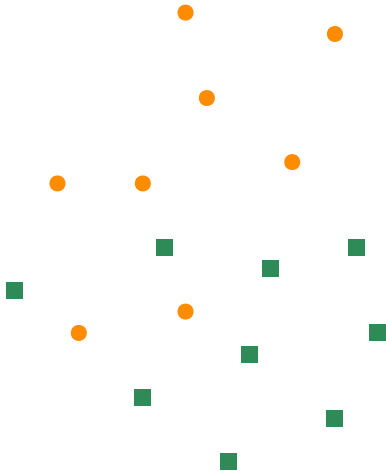
Następnie punktowi x przypisujemy klasę, do której należy najwięcej „sąsiadów”.

Wybór metryki jest dowolny, najczęściej jednak przyjmuje się metrykę euklidesową:

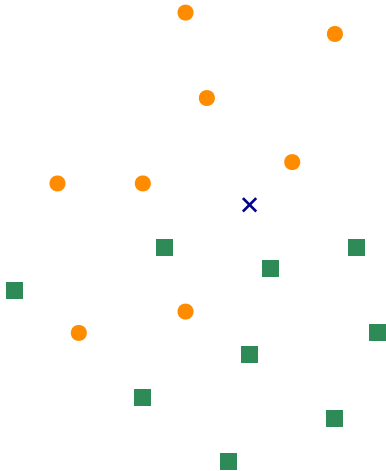
$$d(x^{(i)}, x^{(j)}) = \sqrt{\sum_{n=1}^d (x_n^{(i)} - x_n^{(j)})^2}.$$

Inne wykorzystywane metryki to na przykład odległość Hamminga lub ℓ^p .

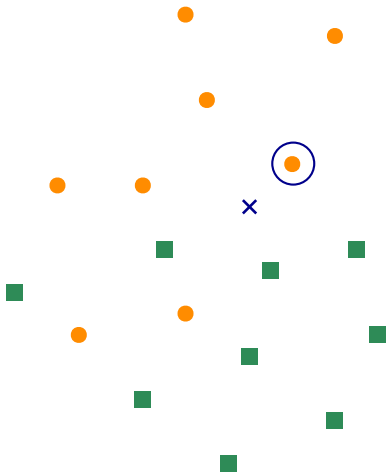
Przykład: $k = 1$



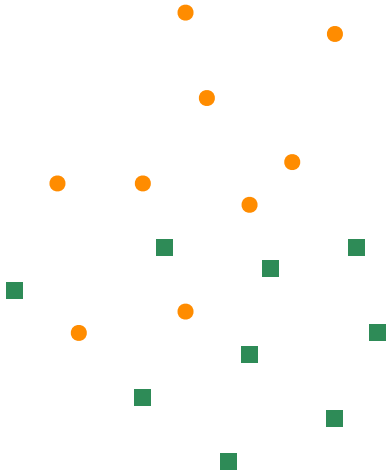
Przykład: $k = 1$



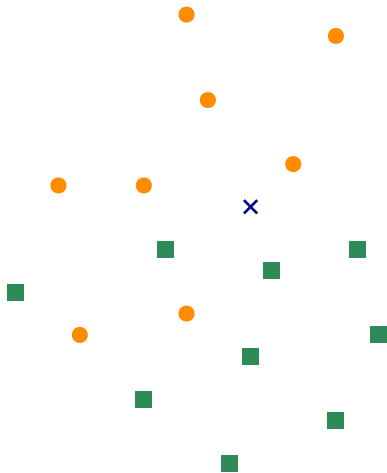
Przykład: $k = 1$



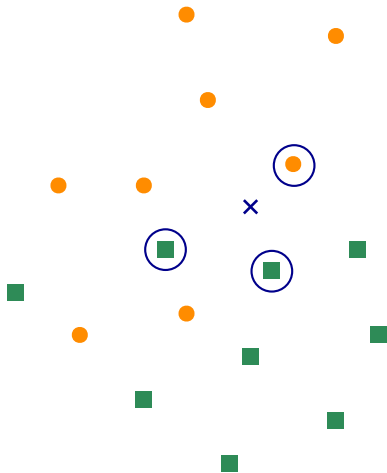
Przykład: $k = 1$



Przykład: $k = 3$



Przykład: $k = 3$



Przykład: $k = 3$

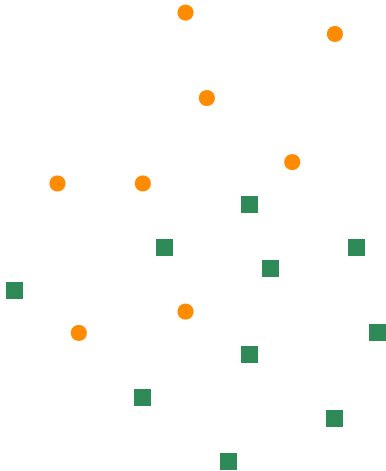
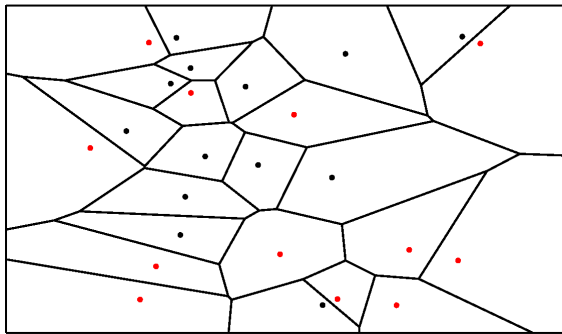


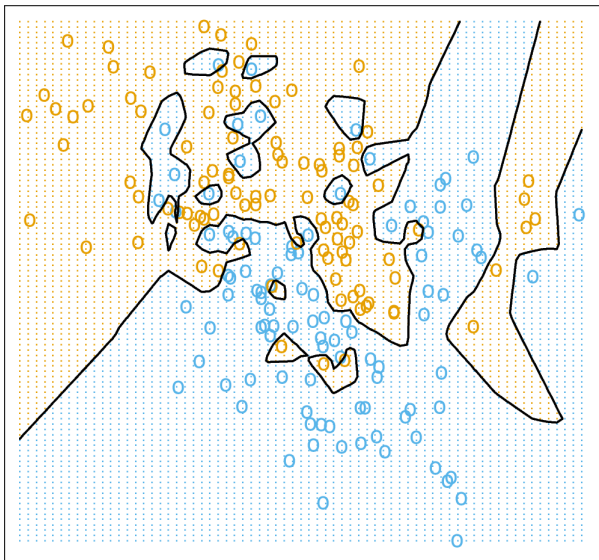
Diagram Woronoja

Niech $X \subset Y$ będzie zbiorem zawartym w zbiorze Y , na którym dana jest metryka d .
Komórką Woronoja elementu $x \in X$ nazywamy zbiór wszystkich punktów $z \in Y$, dla których punkt x jest najbliższym punktem z X .

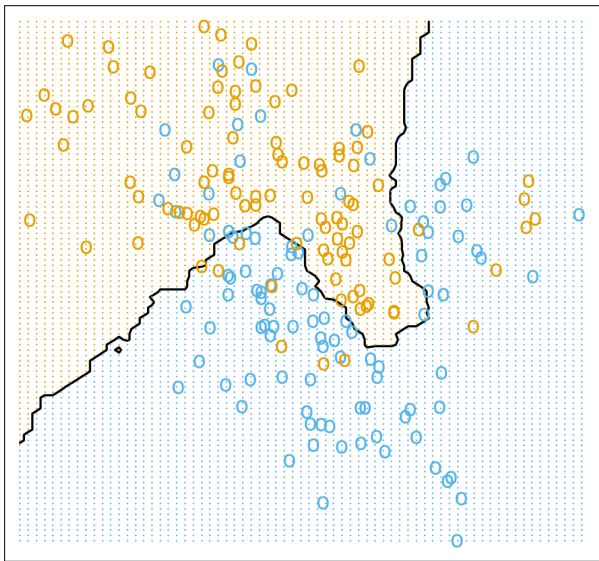
$$\text{Vor}_X(x) = \{y \in Y: d(y, x) \leq d(y, z) \text{ dla wszystkich } z \in X\}.$$



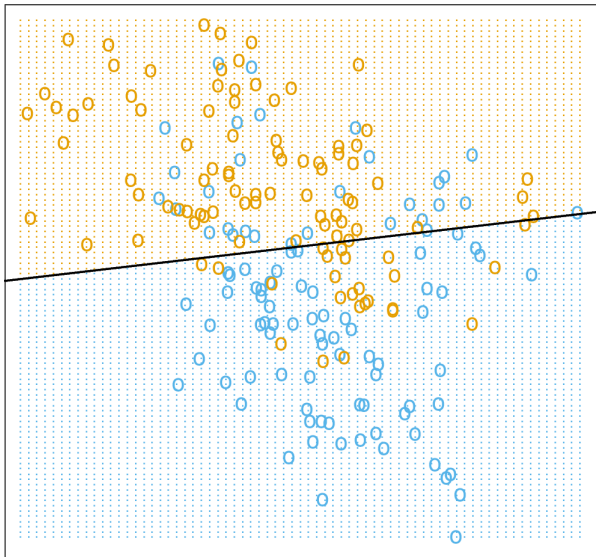
Granica decyzyjna: $k = 1$



Granica decyzyjna: $k = 15$



Granica decyzyjna: regresja liniowa



Ilu sąsiadów wybrać? Jak rozwiązać problemy z kNN?

Jak wybrać wartość k ?

- Przy małej wartości k szum może wpływać na wynik.
- Duża wartość k oznacza duży koszt obliczeniowy.
- Korzystamy ze zbioru walidacyjnego.
- W przypadku parzystej liczby klas wybiera się k nieparzyste.
- Przyjmuje się, że $k \leq \sqrt{m}$ (m to moc zbioru treningowego).

Ilu sąsiadów wybrać? Jak rozwiązać problemy z kNN?

Jak wybrać wartość k ?

- Przy małej wartości k szum może wpływać na wynik.
- Duża wartość k oznacza duży koszt obliczeniowy.
- Korzystamy ze zbioru walidacyjnego.
- W przypadku parzystej liczby klas wybiera się k nieparzyste.
- Przyjmuje się, że $k \leq \sqrt{m}$ (m to moc zbioru treningowego).

Duże wymagania pamięciowe. Złożoność obliczeniowa rzędu $\mathcal{O}(kdm)$. **Co robić?**

- Wybrać mniej cech.
- Wykorzystać np. kd -drzewa do przechowywania posortowanych danych treningowych.
- Obliczyć przybliżoną odległość (locality-sensitive hashing).
- Wyeliminować redundancję.

Wady i zalety kNN

Zalety:

- brak założeń o danych,
- prostota,
- wysoka dokładność,
- wykorzystanie w problemach regresji i klasyfikacji.

Wady i zalety kNN

Zalety:

- brak założeń o danych,
- prostota,
- wysoka dokładność,
- wykorzystanie w problemach regresji i klasyfikacji.

Wady:

- wysoki koszt obliczeń,
- przechowywanie wszystkich danych treningowych,
- uwzględnianie mało istotnych cech.

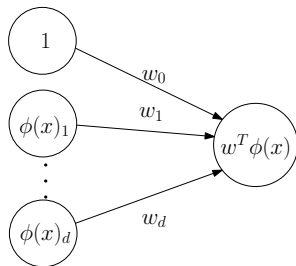
Sieci neuronowe

Przestrzeń hipotez nieliniowych

Rozważając modele liniowe braliśmy pod uwagę hipotezy postaci $w^T \phi(x)$ dla danej wejściowej $x \in \mathbb{R}^k$ i wektora cech $\phi: \mathbb{R}^k \rightarrow \mathbb{R}^d$.

Przestrzeń hipotez nieliniowych

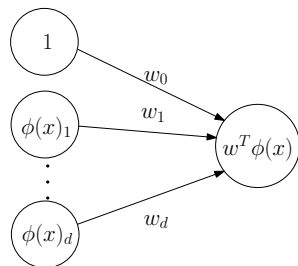
Rozważając modele liniowe braliśmy pod uwagę hipotezy postaci $w^T \phi(x)$ dla danej wejściowej $x \in \mathbb{R}^k$ i wektora cech $\phi: \mathbb{R}^k \rightarrow \mathbb{R}^d$.



Przestrzeń hipotez nieliniowych

Rozważając modele liniowe braliśmy pod uwagę hipotezy postaci $w^T \phi(x)$ dla danej wejściowej $x \in \mathbb{R}^k$ i wektora cech $\phi: \mathbb{R}^k \rightarrow \mathbb{R}^d$.

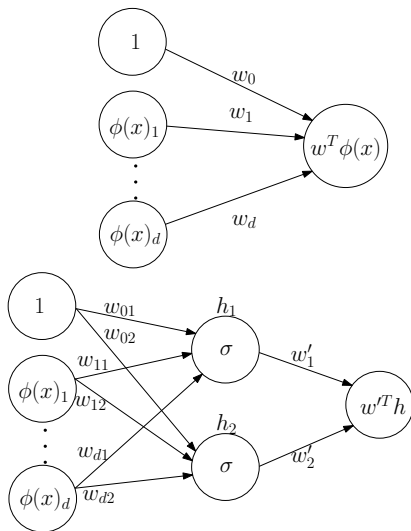
W podstawowej wersji sieci neuronowych
(*wielowarstwowym perceptronie*)
pojawiają się dodatkowe warstwy.



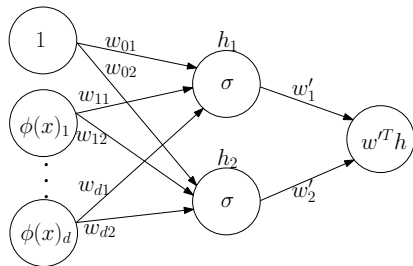
Przestrzeń hipotez nieliniowych

Rozważając modele liniowe braliśmy pod uwagę hipotezy postaci $w^T \phi(x)$ dla danej wejściowej $x \in \mathbb{R}^k$ i wektora cech $\phi: \mathbb{R}^k \rightarrow \mathbb{R}^d$.

W podstawowej wersji sieci neuronowych (*wielowarstwowym perceptronie*) pojawiają się dodatkowe warstwy.



Wielowarstwowy perceptron



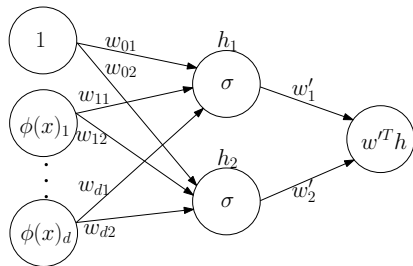
Dla wektora $w_j \in \mathbb{R}^d$ mamy

$$h_j(x) = \sigma(w_j^T \phi(x)),$$

a dla wektora $w' \in \mathbb{R}^2$

$$w'^T h = w'_1 \sigma(w_1^T \phi(x)) + w'_2 \sigma(w_2^T \phi(x)).$$

Wielowarstwowy perceptron



Dla wektora $w_j \in \mathbb{R}^d$ mamy

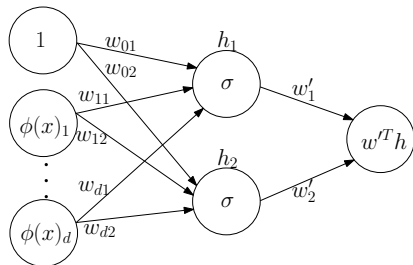
$$h_j(x) = \sigma(w_j^T \phi(x)),$$

a dla wektora $w' \in \mathbb{R}^2$

$$w'^T h = w'_1 \sigma(w_1^T \phi(x)) + w'_2 \sigma(w_2^T \phi(x)).$$

W powyższym przykładzie mamy jedną *ukrytą warstwę*, która składa się z dwóch węzłów. W ogólności możemy mieć dowolnie wiele warstw, z których każda może mieć dowolną liczbę węzłów.

Wielowarstwowy perceptron



Dla wektora $w_j \in \mathbb{R}^d$ mamy

$$h_j(x) = \sigma(w_j^T \phi(x)),$$

a dla wektora $w' \in \mathbb{R}^2$

$$w'^T h = w'_1 \sigma(w_1^T \phi(x)) + w'_2 \sigma(w_2^T \phi(x)).$$

W powyższym przykładzie mamy jedną *ukrytą warstwę*, która składa się z dwóch węzłów. W ogólności możemy mieć dowolnie wiele warstw, z których każda może mieć dowolną liczbę węzłów.

Funkcję $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ nazywamy *funkcją aktywacyjną*. W praktyce najczęściej stosuje się tgh (tangens hiperboliczny) lub ReLU (*rectified linear unit*): $\text{ReLU}(x) = \max(0, x)$.

Regresja na przykładzie

Aby zbudować intuicję, rozważmy przykład prostego problemu regresji o przestrzeni wejść $\mathcal{X} = \mathbb{R}$ oraz przestrzeni wyjść $\mathcal{Y} = \mathbb{R}$.

Jako przestrzeń hipotez przyjmujemy sieć neuronową z jedną ukrytą warstwą z trzema węzłami:

$$\mathcal{H} = \{h: \mathbb{R} \rightarrow \mathbb{R}: h(x) = w_0 + w_1 h_1(x) + w_2 h_2(x) + w_3 h_3(x)\},$$

gdzie $h_j(x) = \sigma(v_j x + b_j)$ dla $j = 1, 2, 3$ oraz $\sigma = \text{tgh}$.

Regresja na przykładzie

Aby zbudować intuicję, rozważmy przykład prostego problemu regresji o przestrzeni wejść $\mathcal{X} = \mathbb{R}$ oraz przestrzeni wyjść $\mathcal{Y} = \mathbb{R}$.

Jako przestrzeń hipotez przyjmujemy sieć neuronową z jedną ukrytą warstwą z trzema węzłami:

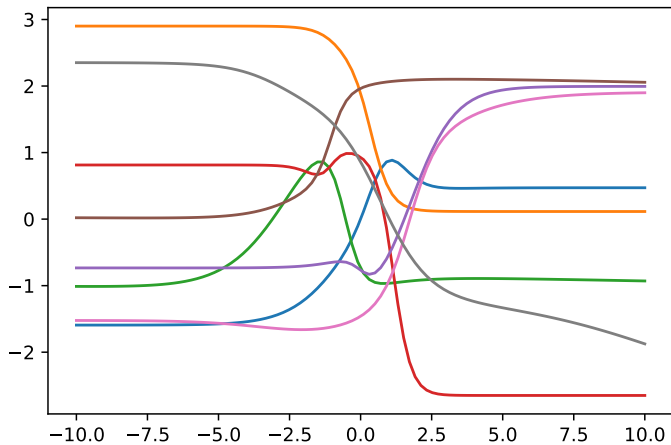
$$\mathcal{H} = \{h: \mathbb{R} \rightarrow \mathbb{R}: h(x) = w_0 + w_1 h_1(x) + w_2 h_2(x) + w_3 h_3(x)\},$$

gdzie $h_j(x) = \sigma(v_j x + b_j)$ dla $j = 1, 2, 3$ oraz $\sigma = \text{tgh}$.

Musimy wyznaczyć 10 parametrów rzeczywistych:

$$v_1, v_2, v_3, b_1, b_2, b_3, w_0, w_1, w_2, w_3.$$

Regresja na przykładzie



Przykładowe osiem hipotez dla losowych wyborów parametrów $v_1, v_2, v_3, b_1, b_2, b_3, w_0, w_1, w_2, w_3$.

Regresja na przykładzie

Chcemy wybrać hipotezę, która minimalizuje ryzyko empiryczne. Przyjmijmy kwadratową funkcję błędu.

Dla zbioru treningowego $\{(x^{(i)}, y^{(i)}) : i = 1, \dots, m\}$ szukamy

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^{10}} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2,$$

gdzie $\theta = (v_1, v_2, v_3, b_1, b_2, b_3, w_0, w_1, w_2, w_3)$.

Regresja na przykładzie

Chcemy wybrać hipotezę, która minimalizuje ryzyko empiryczne. Przyjmijmy kwadratową funkcję błędu.

Dla zbioru treningowego $\{(x^{(i)}, y^{(i)}) : i = 1, \dots, m\}$ szukamy

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^{10}} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2,$$

gdzie $\theta = (v_1, v_2, v_3, b_1, b_2, b_3, w_0, w_1, w_2, w_3)$.

Tym razem funkcja błędu nie jest wypukła ze względu na parametry, więc nie mamy metody gwarantującej znalezienie minimum globalnego.

Regresja na przykładzie

Chcemy wybrać hipotezę, która minimalizuje ryzyko empiryczne. Przyjmijmy kwadratową funkcję błędu.

Dla zbioru treningowego $\{(x^{(i)}, y^{(i)}) : i = 1, \dots, m\}$ szukamy

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^{10}} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2,$$

gdzie $\theta = (v_1, v_2, v_3, b_1, b_2, b_3, w_0, w_1, w_2, w_3)$.

Tym razem funkcja błędu nie jest wypukła ze względu na parametry, więc nie mamy metody gwarantującej znalezienie minimum globalnego.

W praktyce minimum lokalne jest często wystarczające.

Regresja na przykładzie

Chcemy wybrać hipotezę, która minimalizuje ryzyko empiryczne. Przyjmijmy kwadratową funkcję błędu.

Dla zbioru treningowego $\{(x^{(i)}, y^{(i)}) : i = 1, \dots, m\}$ szukamy

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^{10}} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2,$$

gdzie $\theta = (v_1, v_2, v_3, b_1, b_2, b_3, w_0, w_1, w_2, w_3)$.

Tym razem funkcja błędu nie jest wypukła ze względu na parametry, więc nie mamy metody gwarantującej znalezienie minimum globalnego.

W praktyce minimum lokalne jest często wystarczające.

Ponieważ funkcja błędu jest różniczkowalna, to stosując np. metody gradientowe możemy znaleźć minimum lokalne.

Funkcja aktywacyjna: tylko nie wielomian

Twierdzenie (Leshno, Lin, Pinkus, Schocken)

Niech $f: A \rightarrow \mathbb{R}$ będzie funkcją ciągłą na zwartym zbiorze $A \subseteq \mathbb{R}^n$, a $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ lokalnie ograniczoną funkcją kawałkami ciągłą.

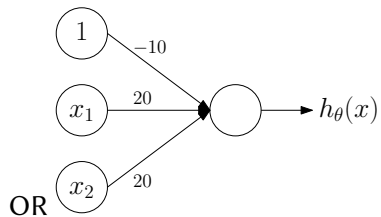
Dla dowolnego $\varepsilon > 0$ istnieje $N \in \mathbb{N}$ oraz parametry $v_j, b_j \in \mathbb{R}$ i $w_j \in \mathbb{R}^n$ takie, że dla funkcji

$$F(x) = \sum_{j=1}^N v_j \sigma(w_j^T x + b_j)$$

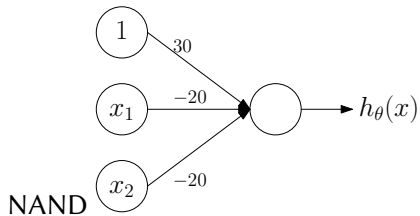
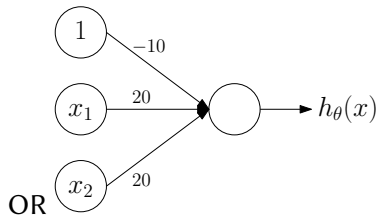
zachodzi $|F(x) - f(x)| < \varepsilon$ dla każdego $x \in A$ wtedy i tylko wtedy, gdy σ nie jest wielomianem.

Oznacza to, że sieć neuronowa z tylko jedną (być może bardzo dużą) warstwą ukrytą może z wybraną dokładnością przybliżać dowolną funkcję ciągłą określoną na zbiorze zwartym, o ile tylko funkcja aktywacyjna nie jest wielomianem.

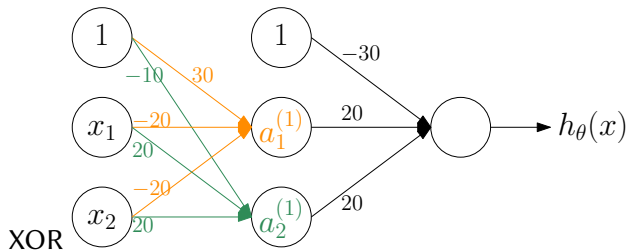
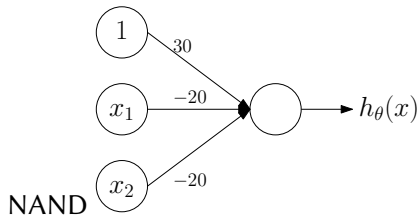
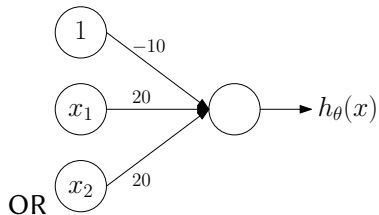
Sieć neuronowa dla klasyfikacji: przykład



Sieć neuronowa dla klasyfikacji: przykład

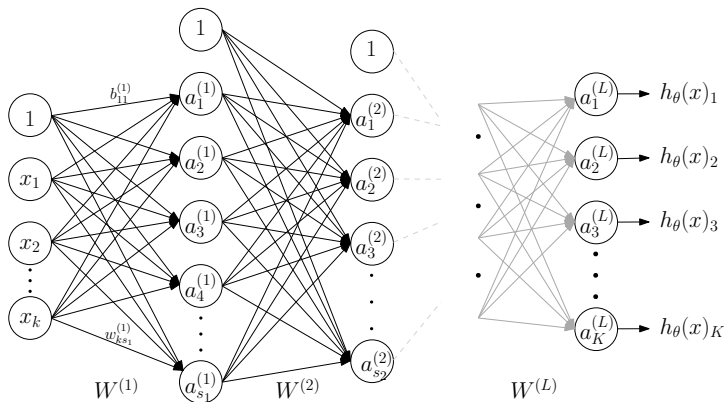


Sieć neuronowa dla klasyfikacji: przykład



Sieć neuronowa dla klasyfikacji wieloklasowej

Rozważmy problem klasyfikacji z przestrzenią wejść $\mathcal{X} = \mathbb{R}^k$ i wyjść $\mathcal{Y} = \{1, \dots, K\}$. Budujemy sieć z L warstwami (poza wejściową). Warstwa $\ell \in \{1, \dots, L-1\}$ ma $s_\ell + 1$ węzłów (dodatkowy węzeł odpowiada wyrazowi wolnemu). W warstwie L (wyjściowej) mamy K węzłów.



Trochę oznaczeń

Parametrami w tak rozważanym modelu są elementy macierzy $W^{(\ell)}$ i wektorów $b^{(\ell)}$:

$$W^{(\ell)} = \begin{bmatrix} w_{11}^{(\ell)} & \cdots & w_{s_{\ell-1}1}^{(\ell)} \\ \vdots & \ddots & \vdots \\ w_{1s_{\ell}}^{(\ell)} & \cdots & w_{s_{\ell-1}s_{\ell}}^{(\ell)} \end{bmatrix}, \quad b^{(\ell)} = \begin{bmatrix} b_1^{(\ell)} \\ \vdots \\ b_{s_{\ell}}^{(\ell)} \end{bmatrix}.$$

Niech $\theta := (w_{11}^{(1)}, \dots, w_{s_{L-1}K}^{(L)}, b_1^{(1)}, \dots, b_K^{(L)})$.

Definiujemy $a^{(0)} := x$ oraz $a^{(\ell)} := \sigma(W^{(\ell)} a^{(\ell-1)} + b^{(\ell)})$ dla $\ell = 1, \dots, L$.

Dla uproszczenia zdefiniujmy $z^{(\ell)} := W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$. Wtedy $a^{(\ell)} = \sigma(z^{(\ell)})$.

Ponownie softmax

Chcemy znaleźć taką hipotezę h_θ , aby $h_\theta(x) \approx e_j$, gdzie $e_j \in \mathbb{R}^K$ jest wektorem jednostkowym. Wówczas zdecydujemy, że wejściowy x należy do klasy j .

Wykorzystamy funkcję softmax $\mathbf{s}: \mathbb{R}^K \rightarrow \mathbb{R}^K$:

$$\mathbf{s}(z) = \left[\frac{\exp(z_1)}{\sum_{j=1}^K \exp(z_j)}, \dots, \frac{\exp(z_K)}{\sum_{j=1}^K \exp(z_j)} \right]^T.$$

Ostatecznie hipotezę $h_\theta: \mathbb{R}^k \rightarrow \mathbb{R}^K$ możemy zapisać następująco:

$$h_\theta(x) = \mathbf{s}(a^{(L)}).$$

Funkcja błędu

Dla zbioru treningowego $\{(x^{(i)}, y^{(i)}): i = 1, \dots, m\}$ rozważymy funkcję kosztu

$$\mathcal{J}(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_j^{(i)} \ln (h_{\theta}(x^{(i)})_j).$$

Możemy dodać jeszcze składnik regularyzacji:

$$\frac{\lambda}{2m} \sum_{\ell=1}^{L-1} \sum_{i=1}^{s_{\ell}} \sum_{j=1}^{s_{\ell+1}} (w_{ji}^{(\ell)})^2.$$

Celem jest wyznaczenie $\hat{\theta} = \arg \min_{\theta} \mathcal{J}(\theta)$. Musimy wyznaczyć $\mathcal{J}(\theta)$, a następnie $\frac{\partial}{\partial w_{rj}^{(\ell)}} \mathcal{J}(\theta)$ i $\frac{\partial}{\partial b_j^{(\ell)}} \mathcal{J}(\theta)$ dla $\ell = 1, \dots, L$ oraz odpowiednich r, j .

Do tego służy algorytm *propagacji wstecznej* (backpropagation).

Propagacja wsteczna: bliżej formalizacji

- Dla próbki $(x^{(i)}, y^{(i)})$ ustaw wartość warstwy wejściowej $a^{(0)}$.
- Dla kolejnych warstw $\ell = 1, \dots, L$ oblicz $\sigma(z^{(\ell)})$ i przypisz do $a^{(\ell)}$.
- Oblicz wektor błędów $\delta^{(L)}$ dla ostatniej warstwy.
- Dla kolejnych warstw $\ell = L - 1, \dots, 1$ oblicz wektory błędów $\delta^{(\ell)}$.
- Gradient funkcji błędu jest wektorem pochodnych cząstkowych

$$\frac{\partial \mathcal{J}}{\partial w_{rj}^{(\ell)}} = a_r^{(\ell-1)} \delta_j^{(\ell)},$$

$$\frac{\partial \mathcal{J}}{\partial b_j^{(\ell)}} = \delta_j^{(\ell)}.$$

Propagacja wsteczna: algorytm

Rozważmy jednoelementowy zbiór treningowy $\{(x^{(i)}, y^{(i)})\}$.

$$a^{(0)} \leftarrow x^{(i)} ;$$

for $\ell = 1, \dots, L$ **do**

$$\quad | \quad a^{(\ell)} \leftarrow \sigma(W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}) ;$$

end

$$\mathcal{J} \leftarrow - \sum_{j=1}^K y_j^{(i)} \ln(\mathbf{s}(a^{(L)})_j) ;$$

$$\delta^{(L)} \leftarrow \nabla_{a^{(L)}} \mathcal{J} \odot \sigma'(z^{(L)}) ;$$

for $\ell = L - 1, \dots, 1$ **do**

$$\quad | \quad \delta^{(\ell)} \leftarrow (W^{(\ell+1)})^T \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)}) ;$$

$$\quad \quad \frac{\partial \mathcal{J}}{\partial w_{rj}^{(\ell)}} \leftarrow a_r^{(\ell-1)} \delta_j^{(\ell)} ;$$

$$\quad \quad \frac{\partial \mathcal{J}}{\partial b_j^{(\ell)}} \leftarrow \delta_j^{(\ell)} ;$$

end

Propagacja wsteczna: algorytm

Rozważmy jednoelementowy zbiór treningowy $\{(x^{(i)}, y^{(i)})\}$.

$$a^{(0)} \leftarrow x^{(i)} ;$$

for $\ell = 1, \dots, L$ **do**

$$\quad | \quad a^{(\ell)} \leftarrow \sigma(W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}) ;$$

end

$$\mathcal{J} \leftarrow - \sum_{j=1}^K y_j^{(i)} \ln(s(a^{(L)})_j) ;$$

$$\delta^{(L)} \leftarrow \nabla_{a^{(L)}} \mathcal{J} \odot \sigma'(z^{(L)}) ;$$

for $\ell = L - 1, \dots, 1$ **do**

$$\quad | \quad \delta^{(\ell)} \leftarrow (W^{(\ell+1)})^T \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)}) ;$$

$$\quad \quad \frac{\partial \mathcal{J}}{\partial w_{rj}^{(\ell)}} \leftarrow a_r^{(\ell-1)} \delta_j^{(\ell)} ;$$

$$\quad \quad \frac{\partial \mathcal{J}}{\partial b_j^{(\ell)}} \leftarrow \delta_j^{(\ell)} ;$$

end

Dla większych zbiorów liczymy pochodne cząstkowe dla każdego elementu, sumujemy i dzielimy przez moc zbioru.

Wady i zalety sieci neuronowych

Zalety:

- działają zarówno dla klasyfikacji, jak i regresji,
- sprawdzają się dla danych o bardzo wielu cechach (np. obrazy),
- wyuczona sieć działa szybko,
- liczba warstw i ich rozmiar są dowolne,
- dla dużych danych często dają jedne z najlepszych wyników,
- są zdolne do uogólniania wyuczonej wiedzy.

Wady:

- nie wiemy, co tak naprawdę się dzieje (black box),
- proces uczenia może być bardzo kosztowy obliczeniowo,
- wytrenowany model zależy istotnie od początkowych parametrów.